

Adventure Sample Loop Structure

현재 데모 기준: Title → InGame → Title 루프 완성 상태 / 작성일: 2026-09-05

1. 현재 데모 루프

```
TitleScreen.OnClickStartButton()  
→ Managers.Instance.SceneFlow.StartGame()  
→ SceneFlowManager.TransitionTo(Game, InGame)  
→ Game Scene 로드  
→ AdventureRunManager.StartRun()  
→ GameManager.Reset()  
→ MapManager.GenerateMap()  
→ GridChunkRuntimeGenerator.Generate(settings)  
→ PlayerSpawnManager.Spawn(playerSpawnPosition)  
→ PlayerManager.RegisterPlayer(player)  
→ NotifyDialogue.RegisterDialogue(InGameUI.Dialogue)  
→ CameraManager.BindTarget(player.transform)  
→ LightingManager.PrepareRunLighting()  
→ GoalTrigger 도달 또는 Player 사망  
→ AdventureRunManager.EndRun() / FailRun()  
→ NotifyDialogue.PlayClear() / PlayGameOver()  
→ SceneFlowManager.GoToTitle()
```

2. Manager 클래스

Managers

DDOL 전역 허브입니다. UI, Game, SceneFlow, Map, Camera, Lighting, AdventureRun을 참조하고 PlayerManager를 생성합니다.

사용: 씬이 바뀌어도 유지되는 런타임 진입점입니다.

SceneFlowManager

Title, InGame, Loading 상태와 씬 전환을 담당합니다. Overlay 페이드와 LoadingUI 진행률을 함께 처리합니다.

사용: Title에서 Game 씬을 로드하고, 런 종료 후 Title로 복귀합니다.

GameManager

Playing, Paused, GameOver 상태와 Time.timeScale을 관리합니다.

사용: 런 시작 시 Reset, 실패 시 TriggerGameOver로 게임 오버 상태를 기록합니다.

AdventureRunManager

한 판의 시작, 성공, 실패, 정리를 관리하는 런 오케스트레이터입니다.

사용: 맵 생성, 플레이어 생성, 카메라/조명/다이얼로그 연결, Title 복귀를 조율합니다.

MapManager

런타임 맵 생성과 정리의 외부 진입점입니다.

사용: GridMapGenerationSettings1.asset 기반으로 GridChunk 맵을 생성하고 ClearMap으로 삭제합니다.

PlayerManager

CurrentPlayer, PlayerInput, Inventory, Equipment를 보관합니다.

사용: 런타임 생성 Player를 UI, Dialogue, Interaction에서 참조할 수 있게 합니다.

PlayerSpawnManager

Player prefab 생성, 기존 Player 삭제, PlayerManager 등록/해제를 전담합니다.

사용: AdventureRunManager가 Player 생성 세부사항을 직접 갖지 않게 분리합니다.

CameraManager / CameraFollow2D

Main Camera 확보, Follow 컴포넌트 생성, Player 타겟 바인딩을 담당합니다.

사용: 생성된 Player를 카메라가 따라가도록 연결합니다.

LightingManager

2D Global Light를 보장합니다.

사용: Game 씬에 Light가 없어 화면이 어둡게 보이는 문제를 런 시작 시 보정합니다.

3. Adventure Run 클래스

AdventureRunManager

상태: None, Starting, Playing, Completed, Failed

메서드: StartRun, EndRun, FailRun, ClearRun

EndRun과 FailRun은 내부 FinishRun으로 공통 종료 흐름을 공유합니다. Clear는 Completed, 사망은 Failed로 분리합니다.

GoalTrigger

EndLine Chunk에 배치되는 Goal prefab용 트리거입니다. Touch 또는 Interact 방식으로 도달 처리를 합니다.

결과: Managers.Instance.AdventureRun.EndRun() 호출.

NotifyDialogue

Clear/GameOver 같은 공지성 메시지용 컴포넌트입니다. DialogueDataSO 선택만 담당하고 재생은 DialogueSequencePlayer에 위임합니다.

4. Map Runtime 클래스

GridMapGenerationSettings

GridChunkDatabase, contentChunkCount, maxThreeDirCount, 랜덤 시드, SpawnKind별 프리팹 세트를 보관하는 ScriptableObject입니다.

GridChunkRuntimeGenerator

EditorWindow 없이 런타임에서 Start, Content, SideBranch, End 청크를 생성합니다.

GridChunkGenerationContext

Instantiate/Destroy를 감싼 생성 컨텍스트입니다. 테스트나 확장 시 생성 방식을 교체할 수 있습니다.

RuntimeGridPlacementUtility

청크 연결, 회전, 점유 셀 계산 등 배치 계산을 담당하는 내부 유틸리티입니다.

GridSpawnPrefabSet

SpawnKind와 실제 프리팹 매핑입니다. 현재 데모에서는 Monster 스폰 프리팹 연결에 사용됩니다.

GeneratedMapInfo

생성된 맵 루트에 붙는 결과 정보입니다. playerSpawnPosition과 endPosition을 기록합니다.

5. Player / FSM 연결

PlayerController

GroundedAgentController 기반 Player 허브입니다. Stamina 초기화, PlayerStateFactory 상태 생성, 공격 입력, 사망 완료 이벤트를 처리합니다.

중요 연결: OnDeathFinished에서 AdventureRunManager.FailRun()을 호출합니다.

PlayerInput

Input System 입력을 이동, 점프, 공격, 상호작용 인터페이스로 제공합니다. Dialogue 중 입력 차단 대상입니다.

PlayerStateFactory

Idle, Move, Jump, Fall, Attack, Hit, Death 상태 인스턴스를 구성합니다.

PlayerInteractionHandler

주변 IInteractable을 탐색하고 Interact를 호출합니다. GoalTrigger가 Interact 모드일 때 사용할 수 있습니다.

6. UI / Dialogue 클래스**UIManager**

전역 Overlay와 현재 InGameUI 참조를 관리합니다.

TitleScreen

Start 버튼에서 SceneFlowManager.StartGame을 호출합니다.

UIOverlayController

Fade, Loading, Pause Dim, Damage Flash를 제어합니다.

LoadingUI

LoadSceneAsync 진행률을 Slider/Text로 표시합니다.

InGameUI

HUD, Dialogue, Notification, MenuTabController를 묶는 Game 씬 UI 루트입니다. Awake에서 UIManager에 등록합니다.

UIDialogueController

대화창 열기, 내용 갱신, 단기와 PlayerInput 차단/HUD 표시 전환을 처리합니다.

DialogueSequencePlayer

DialogueDataSO를 순서대로 재생하는 공통 일반 클래스입니다. Play, Advance, Finish 흐름을 공유합니다.

NPCDialogue

NPC 상호작용용 DialogueDataSO 실행 컴포넌트입니다. NotifyDialogue와 같은 재생기를 사용합니다.

**HealthViewComposition /
StaminaViewComposition**

PlayerManager.OnPlayerChanged를 통해 런타임 생성 Player의 Health/Stamina를 HUD에 바인딩합니다.

7. 종료 흐름**클리어**

GoalTrigger → AdventureRunManager.EndRun → NotifyDialogue.PlayClear → SceneFlowManager.GoToTitle
ClearMessage 재생 후 Title 씬으로 복귀합니다.

게임오버

`PlayerController.OnDeathFinished` → `AdventureRunManager.FailRun` → `GameManager.TriggerGameOver` → `NotifyDialogue.PlayGameOver` → `SceneFlowManager.GoToTitle`

GameOver 상태와 시간 정지 처리 후 메시지를 거쳐 Title 씬으로 복귀합니다.

8. 현재 구조 판단

- SceneFlowManager는 씬 이동, AdventureRunManager는 한 판의 시작/종료 흐름을 담당하도록 분리되어 있습니다.
- Player 생성은 PlayerSpawnManager가 전담하므로 AdventureRunManager의 책임이 과도하게 커지지 않습니다.
- 맵 생성은 MapManager와 GridChunkRuntimeGenerator로 분리되어 EditorWindow 없이 런타임에서 동작합니다.
- Dialogue는 DialogueSequencePlayer를 공통 재생기로 두고, 상황별 데이터 선택은 NotifyDialogue와 NPCDialogue가 담당합니다.
- 현재 데모는 Title → InGame → Title 루프 검증용으로 충분하며, 이후 전투/아이템/장비/NPC 루프를 붙이기 좋은 구조입니다.